RAJALAKSHMI ENGINEERING COLLEGE

Rishi K 251001211-2025-IT

R2

Started on	Sunday, 16 August 2026, 10:35 PM
State	Finished
Completed on	Sunday, 16 August 2026, 10:36 PM
Time taken	34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- $n == nums.length$
- $1 <= n <= 5 * 10^4$
- $-2^{31} <= nums[i] <= 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int countInRange(int arr[], int low, int high, int num) {
    int count = 0;
    for (int i = low; i <= high; i++) {
        if (arr[i] == num) {
            count++;
        }
    }
    return count;
}

int majorityElement(int arr[], int low, int high) {
    // Base case: only one element
    if (low == high) {
        return arr[low];
    }

    int mid = low + (high - low) / 2;

    // Divide: find majority element in left and right halves
    int leftMajority = majorityElement(arr, low, mid);
    int rightMajority = majorityElement(arr, mid + 1, high);

    // Combine
    if (leftMajority == rightMajority) {
        return leftMajority;
    }

    int leftCount = countInRange(arr, low, high, leftMajority);
    int rightCount = countInRange(arr, low, high, rightMajority);

    return (leftCount > rightCount) ? leftMajority : rightMajority;
}
```

```
35
36 int main() {
37     int n;
38     scanf("%d", &n);
39
40     int arr[n];
41     for (int i = 0; i < n; i++) {
42         scanf("%d", &arr[i]);
43     }
44
45     int result = majorityElement(arr, 0, n - 1);
46     printf("%d\n", result);
47
48     return 0;
49 }
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course